Distributed PyTorch on the Cyber Innovation Cluster

A New Student's Guide to DDP + SLURM (Multi-Node, Single-GPU Per Node)

Prepared for the Cyber Innovation Lab by Ashim Dahal

Contents

1 Welcome & What This Guide Covers **2**

2 Cluster Overview **2**

3 Set Up Your Environment **2**
3.1 Create and activate a Conda environment . . . 2
3.2 Sanity checks . . . 3

4 DDP Fundamentals in 2 Minutes **3**

5 Code Walkthrough: `train_ddp.py` **3**
5.1 Complete code . . . 3
5.2 Key ideas . . . 6

6 SLURM Job Script Walkthrough **6**
6.1 Complete script . . . 6
6.2 What each part does . . . 7

7 How to Run (overview) **7**

8 How to Run (Thorough) **8**
8.1 Recommended project layout . . . 8
8.2 Upload your code and data . . . 8
8.3 Activate environment and install deps . . . 8
8.4 Set dataset path (easy convention) . . . 9
8.5 Quick single-node sanity check (optional) . . . 9
8.6 Submit a multi-node job with SLURM . . . 9
8.7 Point your training script at the dataset . . . 10

9 Moving Data To/From the Cluster (Cheatsheet) **10**
9.1 Upload (local -> login) . . . 10
9.2 Download (login -> local) . . . 10
9.3 Integrity/space checks . . . 10

10 Storage Hygiene: Clear Caches Regularly **10**
10.1 pip cache . . . 10
10.2 Conda caches and packages . . . 11
10.3 Hugging Face and Torch caches . . . 11
10.4 Keep future cache growth under control (optional) . . . 11
10.5 Final reminders . . . 11

11 Interactive Single-Node Use & Desk Policy **11**

11.1 Interactive: one computer (no SLURM) . . . 11
11.2 Desk-attached machines (c1, c2) . . . 12
11.3 Prefer c3/c4 for multi-node jobs . . . 12
11.4 Passwordless SSH from desks to the login node . . . 12
11.5 Raspberry Pi kiosks (status-only convenience) . . . 12
11.6 Logout policy (important) . . . 13

12 Performance Tuning Cheatsheet **13**

13 Troubleshooting (Most Common Issues) **13**

14 Minimal Diff: Single-GPU ⇒ Multi-Node DDP **13**

15 Appendix A: Helpful SLURM Commands **14**

16 Appendix B: Etiquette & Best Practices **14**

17 Appendix C: Getting Help **14**
17.1 Where to ask . . . 14
17.2 Using ChatGPT responsibly . . . 14

1 Welcome & What This Guide Covers

This document is a practical, end-to-end companion for running Distributed Data Parallel (DDP) PyTorch training on the Cyber Innovation Cluster using SLURM. It pairs the material from your presentation with step-by-step instructions, code walkthroughs, tuning tips, and troubleshooting.

Who is this for?

Absolute beginners to clusters and multi-node training who know basic Python/PyTorch but have never used SLURM or DDP before.

2 Cluster Overview

- **Topology:** One login node and four compute nodes (each with an RTX 5070 Ti and ∼20 CPU cores).
- **Norms:** Do *not* run training on the login node; use it to submit jobs. Create environments in your home folder. Request only the resources you need.
- **Why the cluster?** It is designed for multi-node experiments and typically outperforms common free cloud options.

3 Set Up Your Environment

3.1 Create and activate a Conda environment

1. Load Miniconda (example): `module load miniconda/miniconda3`
2. Create your environment in your home directory: `conda create -p ~/myenv python=3.12`

3. Activate it: `conda activate ~/myenv`
4. Install PyTorch and torchvision: `pip install torch torchvision`

3.2 Sanity checks

- `python -c "import torch; print(torch.cuda.is_available())"` → should print `True` on compute nodes.
- Ensure `nvidia-smi` works on compute nodes.

4 DDP Fundamentals in 2 Minutes

- **World size:** total process count across all nodes.
- **Rank:** process ID in $[0, \texttt{world_size} - 1]$; rank 0 is the *leader*.
- **Local rank:** GPU index on the current node (for one-GPU-per-node, typically 0).
- **Process group:** the communication group created via `dist.init_process_group(...)`.
- **Master address/port:** rendezvous location (usually the first host in the SLURM allocation).

5 Code Walkthrough: `train_ddp.py`

5.1 Complete code

Listing 1: Multi-node DDP training on MNIST (single GPU per node).

```python
#!/usr/bin/env python3
import os, socket, time
from datetime import datetime

import torch
import torch.distributed as dist
import torch.nn as nn
import torch.optim as optim
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, distributed
from torchvision import datasets, transforms

# ---- Speed knobs ----------------------------------------------------------
torch.backends.cudnn.benchmark = True
torch.backends.cuda.matmul.allow_tf32 = True
if hasattr(torch, "set_float32_matmul_precision"):
    torch.set_float32_matmul_precision("high")

def map_slurm_env_if_needed():
    # Map SLURM env to torch.distributed env if torchrun wasn't used
    if "RANK" not in os.environ and "SLURM_PROCID" in os.environ:
        os.environ["RANK"]       = os.environ["SLURM_PROCID"]
        os.environ["WORLD_SIZE"] = os.environ["SLURM_NTASKS"]
        os.environ["LOCAL_RANK"] = os.environ.get("SLURM_LOCALID", "0")
        # Master from first node
```

```python
        if "MASTER_ADDR" not in os.environ:
            os.environ["MASTER_ADDR"] = os.environ.get("SLURM_LAUNCH_NODE_IPADDR",
                                                       socket.gethostbyname(socket.gethostname()))
        os.environ.setdefault("MASTER_PORT", "29500")

def setup():
    map_slurm_env_if_needed()
    dist.init_process_group(backend="nccl", init_method="env://")
    local_rank = int(os.environ.get("LOCAL_RANK", "0"))
    torch.cuda.set_device(local_rank)

def cleanup():
    dist.destroy_process_group()

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 3, 1, 1), nn.ReLU(),
            nn.MaxPool2d(2),                    # 14x14
            nn.Conv2d(32, 64, 3, 1, 0), nn.ReLU(),
            nn.MaxPool2d(2),                    # 6x6
            nn.Flatten(),
            nn.Linear(64 * 6 * 6, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )
    def forward(self, x): return self.net(x)

def best_workers_from_env(default=14):
    # Reserve ~1 core for the main thread
    cpus = int(os.environ.get("SLURM_CPUS_PER_TASK", default + 1))
    return max(1, cpus - 1)

def worker_init_fn(worker_id: int):
    # Cheap, deterministic seeds per worker
    base = 1337 + dist.get_rank() * 1000
    torch.manual_seed(base + worker_id)

def train(epochs=3, batch_size=128):
    setup()

    rank       = dist.get_rank()
    world_size = dist.get_world_size()
    local_rank = int(os.environ.get("LOCAL_RANK", "0"))
    device     = torch.device("cuda", local_rank)

    if rank == 0:
        print(f"[{datetime.now()}] world_size={world_size}  nodes~{world_size}  "
              f"device={torch.cuda.get_device_name(device)}")
    print(f"[{datetime.now()}] Rank {rank}/{world_size} on {socket.gethostname()} using {device}")
```

```python

    # Model / Optimizer / Loss
    model = SimpleCNN().to(device)
    ddp_model = DDP(model, device_ids=[local_rank], output_device=local_rank,
                    gradient_as_bucket_view=True)
    optimizer = optim.Adam(ddp_model.parameters(), lr=1e-3)
    loss_fn   = nn.CrossEntropyLoss()

    # Data
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)),
    ])

    # Only rank 0 downloads; then barrier; then everyone loads without download
    data_root = "./data"
    if rank == 0:
        _ = datasets.MNIST(root=data_root, train=True, transform=transform, download=True)
    dist.barrier()

    dataset = datasets.MNIST(root=data_root, train=True, transform=transform, download=False)
    sampler = distributed.DistributedSampler(dataset, shuffle=True, drop_last=False)

    num_workers      = best_workers_from_env()
    prefetch_factor  = 6  # good starting point for 15 CPU cores
    pin_memory       = True
    persistent       = True

    # Cap prefetch_factor if workers are small
    if num_workers <= 2:
        prefetch_factor = 2

    torch.set_num_threads(num_workers + 1)

    dataloader = DataLoader(
        dataset,
        batch_size=batch_size,
        sampler=sampler,
        num_workers=num_workers,
        pin_memory=pin_memory,
        prefetch_factor=prefetch_factor,
        persistent_workers=persistent,
        # worker_init_fn=worker_init_fn,  # optional: enable deterministic seeding
    )

    # Train
    for epoch in range(epochs):
        sampler.set_epoch(epoch)
```

```
        ddp_model.train()

        epoch_loss = 0.0
        t0 = time.time()
        for step, (images, labels) in enumerate(dataloader):
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            optimizer.zero_grad(set_to_none=True)
            outputs = ddp_model(images)
            loss = loss_fn(outputs, labels)
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item()
            if rank == 0 and step % 50 == 0:
                print(f"[{datetime.now()}] epoch {epoch} step {step} "
                      f"loss {loss:.4f}")

        if rank == 0:
            dt = time.time() - t0
            print(f"[{datetime.now()}] epoch {epoch} done in {dt:.2f}s "
                  f"avg_loss={epoch_loss/max(1, len(dataloader)):.4f}")

    cleanup()

if __name__ == "__main__":
    train()
```

5.2 Key ideas

- **Mapping SLURM → DDP env:** `map_slurm_env_if_needed()` derives `RANK`, `WORLD_SIZE`, `LOCAL_RANK`, and `MASTER_ADDR/PORT` from SLURM variables when you do not launch with `torchrun`.
- **Initialization:** `dist.init_process_group("nccl")` forms the process group; set the CUDA device with `torch.cuda.set_device(local_rank)`.
- **Sharding:** `DistributedSampler` ensures each rank sees a unique shard per epoch; call `sampler.set_epoch(epoch)`.
- **I/O throughput:** tune `num_workers` (from `SLURM_CPUS_PER_TASK`), `prefetch_factor`, `pin_memory`, and use non-blocking transfers.
- **Race-free dataset download:** only rank 0 downloads; then `dist.barrier()`.

6 SLURM Job Script Walkthrough

6.1 Complete script

Listing 2: SLURM script: 4 nodes, 1 task per node, 15 CPUs per task.

```
#!/bin/bash
#SBATCH --job-name=mnist_ddp_4x1
```

```
#SBATCH --partition=main
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=1          # one process per node
#SBATCH --cpus-per-task=15           # 15 CPU cores each
#SBATCH --time=00:30:00
#SBATCH --output=ddp_train.%j.out
#SBATCH --error=ddp_train.%j.err

set -euo pipefail

# If you don't have InfiniBand, keep it off.
export NCCL_DEBUG=INFO
export NCCL_IB_DISABLE=1
# If you need to pin to a specific NIC, uncomment and set the right iface:
# export NCCL_SOCKET_IFNAME=enp1s0

# Use all the CPU threads we requested for PyTorch's internal pools
export OMP_NUM_THREADS="${SLURM_CPUS_PER_TASK}"

# Pick master = first hostname in the allocation
MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n1)
MASTER_PORT=29500
echo "MASTER_ADDR=$MASTER_ADDR MASTER_PORT=$MASTER_PORT"
echo "Nodes: $SLURM_JOB_NODELIST"

# Fan out one task per node; map SLURM env -> torch.distributed env inside each task
srun bash -lc '
  export RANK=$SLURM_PROCID
  export WORLD_SIZE=$SLURM_NTASKS
  export LOCAL_RANK=$SLURM_LOCALID
  export MASTER_ADDR='"$MASTER_ADDR"'
  export MASTER_PORT='"$MASTER_PORT"'
  echo "[$(hostname)] RANK=$RANK LOCAL_RANK=$LOCAL_RANK WORLD_SIZE=$WORLD_SIZE CUDA_VISIBLE_DEVICES=$CUDA_VISIBLE_DEVICES OMP_NUM_THREADS=$OMP_NUM_THREADS"
  python -u train_ddp.py
'
```

6.2 What each part does

- **Resources:** `--nodes=4`, `--ntasks-per-node=1`, `--cpus-per-task=15`.
- **Networking:** `MASTER_ADDR` is the first host in the allocation; if no InfiniBand, set `NCCL_IB_DISABLE=1`.
- **Threading:** `OMP_NUM_THREADS` mirrors `SLURM_CPUS_PER_TASK`.
- **Fan-out:** `srun` launches one process per node and maps SLURM vars to DDP vars for you.

7 How to Run (overview)

1. Upload `train_ddp.py` and `train.sh` to your project directory.
2. Activate your env: `conda activate ~/myenv`

3. Submit the job: `sbatch train.sh`
4. Monitor: `squeue -u $USER`
5. Cancel: `scancel <job_id>`
6. Inspect logs: `less ddp_train.<jobid>.out` and `less ddp_train.<jobid>.err`

8 How to Run (Thorough)

8.1 Recommended project layout

Keep code and data tidy so multi-node jobs can find everything in your shared home. **Example layout**

```
# ASCII-only tree
~/projects/cil-ddp/
|-- train_ddp.py
|-- train.sh
`-- env/            # optional: conda env or env.yml/requirements.txt

~/datasets/
`-- mnist/          # or any other dataset
```

8.2 Upload your code and data

From Linux/macOS (terminal) **Copy code and data to the login node (shared home)**

```
# Code -> project directory
scp -r ./cil-ddp  <your_username>@login:~/projects/

# Data -> shared datasets directory
scp -r ./mnist    <your_username>@login:~/datasets/
```

Resumable sync for big datasets

```
rsync -avhP ./mnist/  <your_username>@login:~/datasets/mnist/
```

From Windows

1. Install `WinSCP`.
2. Host name: `login`, Username: `<your_username>`.
3. Drag and drop your project folder to `~/projects/` and dataset to `~/datasets/`.

8.3 Activate environment and install deps

```
ssh <your_username>@login
conda activate ~/myenv
# (Optional) avoid local pip cache growth during installs:
pip install --no-cache-dir -r requirements.txt
```

8.4 Set dataset path (easy convention)

Most examples assume data under `~/datasets`. You can also make it explicit: **Set a dataset root env var for your jobs**

```
export DATA_ROOT=~/datasets
```

8.5 Quick single-node sanity check (optional)

If you want to test on one GPU interactively before queuing:

```
ssh c1
conda activate ~/myenv
cd ~/projects/cil-ddp
export DATA_ROOT=~/datasets
torchrun --standalone --nproc_per_node=1 train_ddp.py
```

8.6 Submit a multi-node job with SLURM

From the login node:

```
cd ~/projects/cil-ddp
sbatch train.sh
```

Monitor

```
# Your jobs in the queue
squeue -u $USER

# Detailed info for one job
scontrol show job <job_id>

# Live log tail (in another terminal)
tail -f ddp_train.<job_id>.out
```

Cancel / re-run

```
scancel <job_id> # or scancel -u <username>
sbatch train.sh
```

Common run-time checks

```
# On a compute node (via srun or interactive ssh) check GPU/NV driver:
ssh c1 # (or whichever node is shown in your squeue command)
nvidia-smi

# Check that your shared home is visible
ls -lah ~/projects/cil-ddp
```

8.7 Point your training script at the dataset

Add a tiny convenience to your Python so it respects `DATA_ROOT` but still works by default. **Python snippet: dataset root**

```python
import os
DATA_ROOT = os.getenv("DATA_ROOT", os.path.expanduser("~/datasets"))
MNIST_DIR = os.path.join(DATA_ROOT, "mnist")
```

Then use `MNIST_DIR` in your dataset construction:

```python
if rank == 0:
    _ = datasets.MNIST(root=MNIST_DIR, train=True, transform=transform,
    download=True)
dist.barrier()
dataset = datasets.MNIST(root=MNIST_DIR, train=True, transform=transform,
    download=False)
```

9 Moving Data To/From the Cluster (Cheatsheet)

9.1 Upload (local -> login)

```bash
# Small/medium files (simple):
scp -r ./path/to/dir  <user>@login:~/datasets/dir

# Large datasets (resumable + progress):
rsync -avhP ./imagenet/  <user>@login:~/datasets/imagenet/
```

9.2 Download (login -> local)

```bash
scp -r <user>@login:~/projects/cil-ddp/results ./results
```

9.3 Integrity/space checks

```bash
# Disk space on your home mount
df -h ~

# Top space consumers in your home
du -sh ~/* | sort -h

# Size of a dataset
du -sh ~/datasets/imagenet
```

10 Storage Hygiene: Clear Caches Regularly

Shared storage fills quickly. Please clean your own caches after installs and large runs.

10.1 pip cache

```
# See size
du -sh ~/.cache/pip 2>/dev/null || echo "No pip cache"

# Purge
pip cache purge
# Or remove manually if needed:
rm -rf ~/.cache/pip
```

10.2 Conda caches and packages

```
# Show where conda stores pkgs
conda info | grep -i pkgs

# Clean tarballs, pkg caches, and unused pkgs
conda clean --all -y
```

10.3 Hugging Face and Torch caches

```
# Hugging Face hub/models/datasets cache
du -sh ~/.cache/huggingface 2>/dev/null || echo "No HF cache"
rm -rf ~/.cache/huggingface/*

# PyTorch hub (pretrained weights) cache
du -sh ~/.cache/torch 2>/dev/null || echo "No torch cache"
rm -rf ~/.cache/torch/hub/*
```

10.4 Keep future cache growth under control (optional)

Add to your shell init (e.g., /.bashrc) to steer caches to a known path or keep them smaller:

```
# Put HF cache under ~/caches/hf (create it first)
export HF_HOME=~/caches/hf

# Avoid pip cache during installs
alias pipnocache='pip --no-cache-dir'
```

10.5 Final reminders

- Only delete inside your own home (~). Never remove shared system paths.
- After big experiments, archive results you need and clean temp files and caches.

11 Interactive Single-Node Use & Desk Policy

11.1 Interactive: one computer (no SLURM)

If you just need one machine, you can SSH directly to a compute node and run interactively.

Listing 3: Single-node run (one GPU, no SLURM).

```
# From your laptop or campus machine:
```

```
ssh c1

# On c1:
conda activate ~/myenv

# Run a DDP-aware script on a single GPU:
torchrun --standalone --nproc_per_node=1 train_ddp.py

# (Or run a non-DDP script with plain `python`.)
```

Why `torchrun`? The training script expects Distributed Data Parallel (DDP) variables. `torchrun -standalone` sets up a local, single-process process group automatically, so it works for one GPU without extra env setup.

11.2 Desk-attached machines (`c1`, `c2`)

Nodes `c1` and `c2` are also connected to desks for walk-up, interactive work. Please:

- Use `c1`/`c2` when you are physically at the desks and need a local session.
- For multi-node or queued jobs, prefer `c3` and `c4` (see below).

11.3 Prefer `c3`/`c4` for multi-node jobs

When you submit jobs via SLURM, it helps availability if you steer larger runs to non-desk nodes. You can exclude desk machines or target others explicitly:

Listing 4: Steer jobs away from desk nodes.

```
# Option A: exclude c1 and c2
#SBATCH --exclude=c1,c2

# Option B: explicitly list non-desk nodes (example)
#SBATCH --nodelist=c3,c4
```

11.4 Passwordless SSH from desks to the login node

All machines are configured for keyless (passwordless) SSH within the cluster. If you are working at the `c1` desk and decide to submit a multi-node job:

Listing 5: Submit SLURM from a desk-attached node.

```
# On c1's terminal (while at the desk):
ssh login      # passwordless hop to the login node
cd ~/your/project
sbatch train.sh
```

This works seamlessly because **all compute nodes share your home directory via the login node**. Your scripts and data are visible cluster-wide.

11.5 Raspberry Pi kiosks (status-only convenience)

If you just need to check jobs, we provide Raspberry Pi kiosks. Use them to SSH into the login node and check queue status or view logs:

Listing 6: Quick status checks from a Raspberry Pi.

```
ssh login
squeue -u $USER
tail -f ddp_train.<jobid>.out
```

11.6 Logout policy (important)

Except for the Raspberry Pi kiosks, **please log out of any machine after you are done**. The cluster uses passwordless SSH inside the fabric, so logging out prevents accidental reuse of your session and keeps resources available for others.

12 Performance Tuning Cheatsheet

- Scale `num_workers` up to `SLURM_CPUS_PER_TASK - 1`; try `prefetch_factor` in $[2, 8]$.
- Enable `pin_memory=True` and `non_blocking=True`.
- Use `torch.backends.cudnn.benchmark=True` and TF32 (`allow_tf32`) when acceptable.
- Consider `gradient_as_bucket_view=True` and tuning `bucket_cap_mb`.
- Keep `find_unused_parameters=False` (default) for static graphs.

13 Troubleshooting (Most Common Issues)

Hangs at init / timeout All tasks must resolve `MASTER_ADDR`; ensure the port is open, firewalls are permissive, and PyTorch/CUDA are compatible across nodes.

NCCL errors No InfiniBand? Keep `NCCL_IB_DISABLE=1`. Specify NIC with `NCCL_SOCKET_IFNAME` (e.g., `enp1s0`) if needed.

"Address already in use" Stale processes left on a node. Cancel the job and kill leftover Python processes.

Inconsistent environments Use a shared home or identically provisioned envs. Paths should match on all nodes.

Dataset races Only rank 0 downloads. Others should not pass `download=True`. Barrier afterward (already in code).

Low GPU utilization Increase batch size, raise `num_workers`/`prefetch_factor`, verify `pin_memory`.

14 Minimal Diff: Single-GPU ⇒ Multi-Node DDP

```
- model = Net().to(device)
+ model = Net().to(device)
+ model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[
    local_rank])

- loader = DataLoader(dataset, batch_size=..., shuffle=True)
+ sampler = torch.utils.data.distributed.DistributedSampler(dataset,
    shuffle=True)
```

```
+ loader  = DataLoader(dataset, batch_size=..., sampler=sampler, ...)

+ dist.init_process_group("nccl", init_method="env://")
+ torch.cuda.set_device(local_rank)

 for epoch in range(E):
+   sampler.set_epoch(epoch)
    ...
```

15 Appendix A: Helpful SLURM Commands

- `sbatch train.sh` submit a job
- `squeue -u $USER` list your jobs
- `scancel <job_id>` cancel a job
- `sinfo -Nel` detailed node info (if permitted)

16 Appendix B: Etiquette & Best Practices

- Never train on the login node; use it only to submit jobs and do lightweight tasks.
- Keep experiments reproducible (`requirements.txt` or `environment.yml`).
- Request only the resources you need and release them promptly.

17 Appendix C: Getting Help

17.1 Where to ask

- **Quick questions:** ask your lab mates or TA during lab hours.
- **Cluster issues (accounts, nodes, queues):** contact the lab admin.
- **Docs and cheatsheets:** see this guide and the examples in `~/projects/cil-ddp`.

17.2 Using ChatGPT responsibly

This is real-world work, not a classroom. Feel free to use ChatGPT to draft scripts, debug errors, or clarify concepts.

- Paste exact error messages and relevant snippets (avoid secrets and private keys).
- Ask for minimal, copy-pasteable fixes and explanations.
- Validate solutions on a small test before running large cluster jobs.